Skill progress monitoring

Catching robot failures while they happen

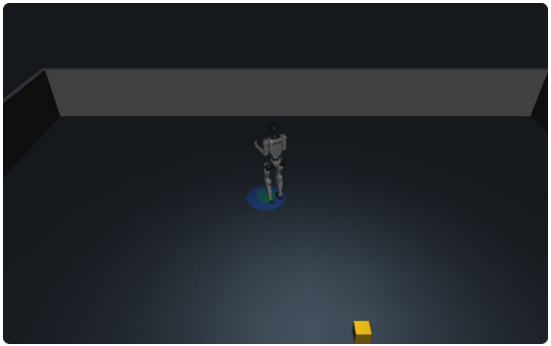
Pinhas Nehoray Aburmad

Ben-Gurion University of the Negev
Supervisor: Prof. Ronen Brafman



The problem

- Robots often fail **in the middle** of a task.
- They often **do not notice**.
- They report “done” anyway.
- **Goal:** notice the failure **the moment it happens**.



The robot says “done” at the drop-off (blue). The box (yellow) is still on the floor.

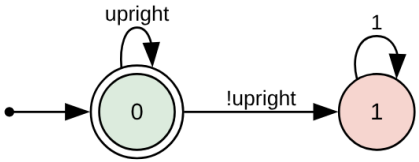
Related work

Work	What it does	What is missing
Runtime verification of LTL [Bauer 2011]	checks rules on a running system	no task stages: “eventually” says nothing until the end
Robot monitors for ROS [Ferrando 2020; Yamaguchi 2024]	check rules on robot messages	rules written by hand; no task progress
Monitoring with a hidden state [Stoller 2011]	guesses the hidden state with a probability model	needs that model; no task stages
English $\rightarrow$ logic [Liu 2023; Cosler 2023; Chen 2023]	turn words into LTL rules	used to plan, not to watch a running task
Learned failure detectors [Lin 2025; Zhang 2026; Kapoor 2026]	learn failures from camera data	need training data; hard to say why it fired

Our gap: rules made from plain words **+** task stages **+** only the robot’s own sensors.

Background: what is a monitor?

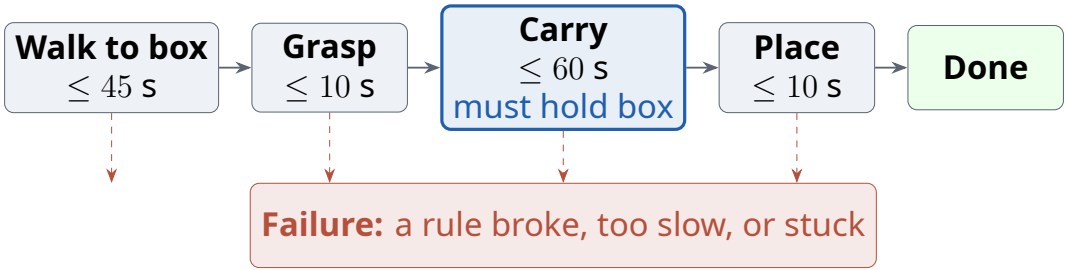
- A program that **watches** the robot's messages.
- It checks **rules over time**, written in **LTL** (a logic of time).
- Example: "*always* stay upright".
- Each rule becomes a small **state machine**.
- Problem: "*eventually* deliver" **never fails early**.



"Always upright" as a state machine. One fall moves it to state 1 (red), forever.

Our idea: the stages of the task

Each stage has its own rules and its own time limit.



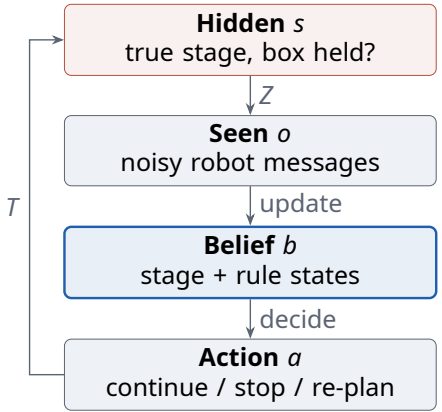
While carrying, the robot **must hold the box**. A drop is caught **at once**.

The mathematical model: a POMDP

- **Hidden** s: what really happens (stage, box held?).
- **Seen** o: the robot's messages.
- **Belief** b: our best guess of s.
- **Action** a: continue, stop, or re-plan.

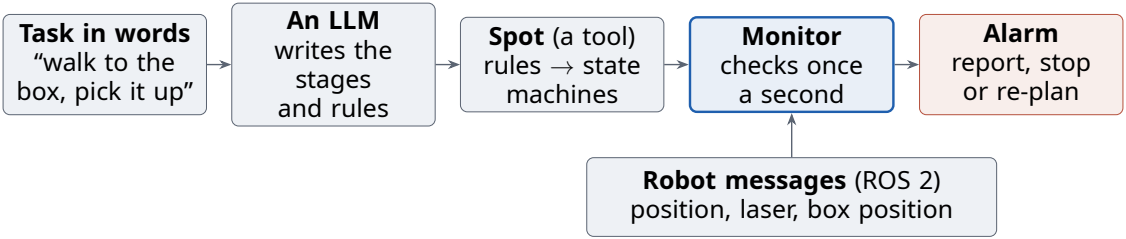
$$b'(s') \;\propto\; \underbrace{Z(o \mid s')}_{\text{does } o \text{ fit } s'?} \sum_{s} \underbrace{T(s' \mid s, a)}_{\text{how } s \text{ changes}} \; b(s)$$

Today: the guess is one point (stage + rule states); the update is one rule step.



POMDP [Kaelbling 1998]. Why stages help: "not holding" is fine while walking, a failure while carrying.

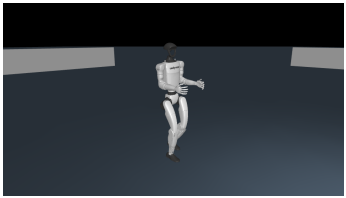
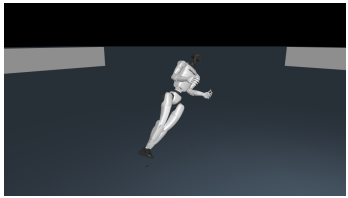
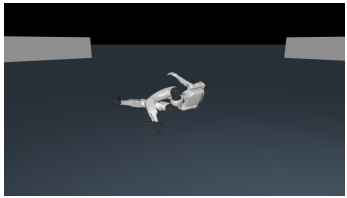
Method: how the monitor is built and run



- The rules are written **automatically**, from the task in words.
- The monitor uses **only sensors**. It never asks the planner.

Test bed: a humanoid robot in a simulator

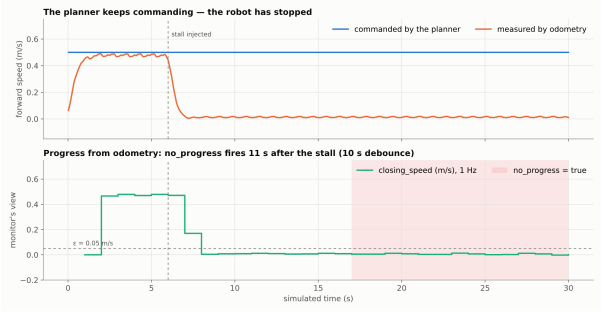
- Unitree **G1** robot in the **MuJoCo** physics simulator.
- We **cause failures on purpose**: push, stall, drop, failed grasp.
- We compare the monitor **with stages** and **without**.

Walking → pushed → fallen. The simulator knows the truth; the monitor does not.

Experiment 1: a short walk (1800 runs)

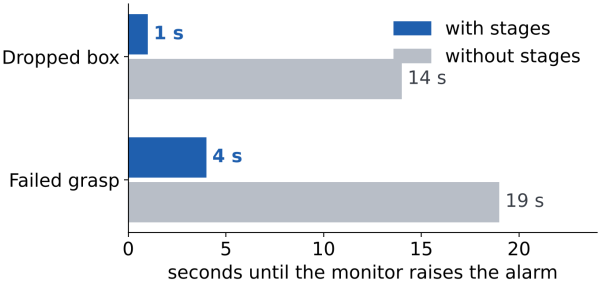
- Stages helped **little** here: 15 vs. 14 of 15 stalls caught.
- What mattered: measure progress **from the robot's sensors**.
- Without that: only **1 of 15** caught.



The planner still says "go". The robot has stopped. The monitor sees no progress.

Experiment 2: fetch and deliver (3360 runs)

- Four stages.
- Box dropped: caught after **1 s** with stages, 14 s without.
- Grasp failed: **4 s** vs. 19 s.
- Earlier in **all 20** runs.

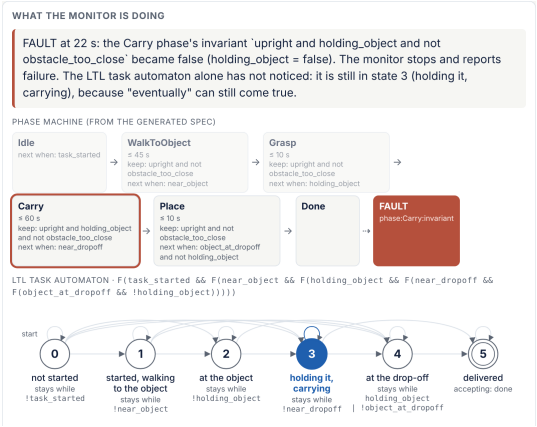


Middle value (median) of 20 runs for each failure.

One run, second by second

- **21.6 s:** the box drops.
- **22 s:** the Carry rule breaks. Alarm.
- The plain LTL rule still waits: “maybe later”.

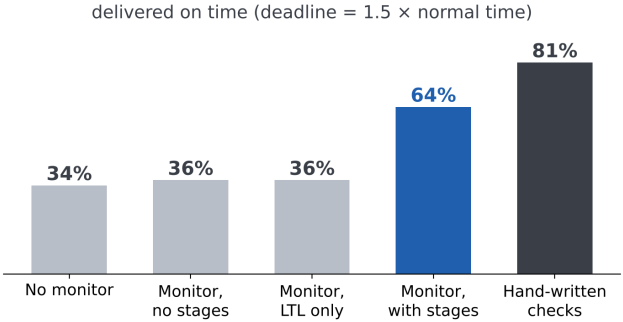
The stage caught it.



From the live demo (one run). The current stage and state are lit.

Does catching early matter?

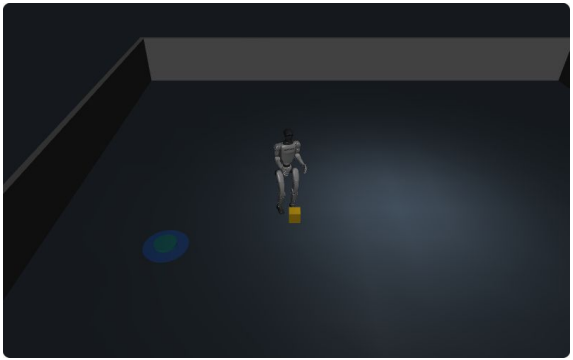
- With no time limit: all monitors ≈ 82–84%.
- With a time limit: **64%** with stages, 36% without.
- Hand-written checks: 81%, but someone must **predict every failure**.



Tasks finished on time, 160 runs per bar. Early knowledge pays when time matters.

Failures nobody planned for

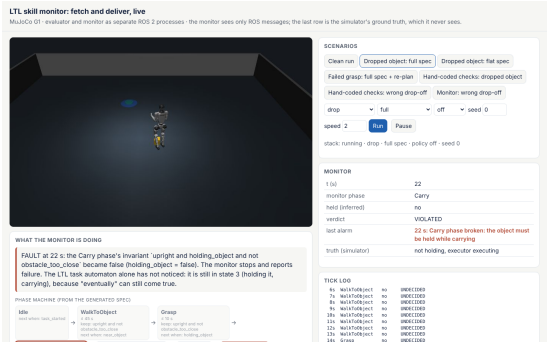
- Box left at the wrong place; robot too close to a wall.
- Hand-written checks: **40 of 40** missed.
- With our monitor: **0 of 40** missed.
- Missed by all: too slow, or a long detour.



The box is far from the drop-off (blue), yet the robot says “done”. Our monitor catches it.

What we learned

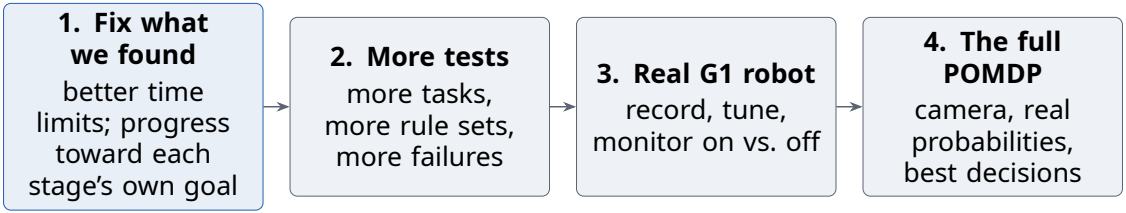
- **Stages catch failures when they happen.**
- Early alarms help **when time matters**.
- Rules come from words, so failures **nobody planned for** are caught too.
- Use the **robot's own sensors**, not the planner.



Live demo, one command:

```
docker run -p 8080:8080
ghcr.io/pinhas019/ltlmonitor-demo
```

To be done



Limits today: the grasp is simulated (no arms on this G1), the camera is simulated, the thresholds are not tuned.

References

Bauer, Leucker, Schallhart. *Runtime Verification for LTL and TLTL.* ACM TOSEM 20, 2011.

Chen, Gandhi, Zhang, Fan. *NL2TL: Transforming Natural Languages to Temporal Logics using Large Language Models.* EMNLP 2023.

Cosler, Hahn, Mendoza, et al. *nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with LLMs.* CAV 2023.

Ferrando, Cardoso, Fisher, Ancona, et al. *ROSMonitoring: A Runtime Verification Framework for ROS.* TAROS 2020.

Kaelbling, Littman, Cassandra. *Planning and Acting in Partially Observable Stochastic Domains.* Artificial Intelligence 101, 1998.

Kapoor, Hammer, Kapoor, et al. *Runtime Monitoring of Perception-Based Autonomous Systems via Embedding Temporal Logic.* arXiv:2605.12651, 2026.

Lin, Duan, Fang, Fox, et al. *FailSafe: Reasoning and Recovery from Failures in Vision-Language-Action Models.* arXiv:2510.01642, 2025.

Liu, Yang, Idrees, Liang, et al. *Grounding Complex Natural Language Commands for Temporal Tasks in Unseen Environments.* CoRL 2023.

Stoller, Bartocci, Seyster, Grosu, Havelund, Smolka, Zadok. *Runtime Verification with State Estimation.* RV 2011.

Yamaguchi, Hoxha, Ničković. *RTAMT: Runtime Robustness Monitors with Application to CPS and Robotics.* STTT 26, 2024.

Zhang, Lu, Wang, Kang, et al. *Foresight: Failure Detection for Long-Horizon Robotic Manipulation with Action-Conditioned World Model Latents.* arXiv:2606.23085, 2026.